

Assignment 4 — Solution Key

Registers, Stack Organization, Instruction Formats, and the Instruction Cycle

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

4.1 Conceptual

Q1. Program-visible registers this course uses: the **general-purpose** register file $R_0, R_1, \dots, R_{n-1}$; the **accumulator** AC (implicit operand/result register of one-address ISAs); and the **stack pointer** SP . **Internal** registers: PC (address of the next instruction), IR (the current instruction), MAR (address for the current memory access), and MDR (data in transit to/from memory). A program can name the program-visible registers because instructions are *defined* to address them as operands/results; the internal registers are never encoded in an instruction field — only the control unit reads and writes them, so the program cannot reference them. The register whose job is only to hold the address of the next instruction is PC .

Q2. Function calls nest deeply (recursion), and each active call needs its own frame — return address, saved registers, local variables. A **register stack** holds only a handful of words (e.g. 64), far too few for deeply nested or recursive calls, so a register stack cannot store the frames. A **memory stack** is as large as main memory, so it can hold arbitrarily many nested frames. Real machines therefore keep the stack in memory with SP in a register: in the memory-stack design SP holds the *address* of the current top of the memory stack; in a register stack, SP is a counter register holding the index/position of the top element within the register file. The trade-off is speed (register stack is fast) against capacity (memory stack is unbounded).

4.2 Numerical

Q3. Using the lecture's convention (contents of register X written $[X]$), $Y = (A - B) \times (C + D)$:

- **3-address:** $SUB\ R1, A, B; ADD\ R2, C, D; MUL\ Y, R1, R2$ — **3 instructions**.
- **2-address:** $MOV\ R1, A; SUB\ R1, B; MOV\ R2, C; ADD\ R2, D; MUL\ Y, R1$ — **5 instructions**. The two MOV s are needed because the destructive form overwrites its first operand ($R1 \leftarrow [R1] - [B]$), so operands must be copied before being modified.
- **1-address (accumulator):** $LOAD\ A; SUB\ B; STORE\ T; LOAD\ C; ADD\ D; MUL\ T; STORE\ Y$ — **7 instructions**. A temporary T in memory is required because the accumulator holds only one partial result at a time.
- **0-address (stack):** $PUSH\ A; PUSH\ B; SUB; PUSH\ C; PUSH\ D; ADD; MUL; POP\ Y$ — **8 instructions**, each the shortest possible.

The four counts: **3, 5, 7, 8** — the same pattern as the lecture’s $(A+B) \times (C+D)$.

Q4. Stack grows downward; top on the right.

Step	Stack after (top right)
initial	[?, 3, 8, 2]
PUSH 5	[?, 3, 8, 2, 5]
ADD	[?, 3, 8, 7] (pop 5, pop 2, push 7)
PUSH 1	[?, 3, 8, 7, 1]
POP	returns 1 $\Rightarrow$ [?, 3, 8, 7]
POP	returns 7 $\Rightarrow$ [?, 3, 8]

Final stack [?, 3, 8]; POPs returned **1** then **7**.

4.3 Design

Q5. To copy the contents of R5 into R3 ($R3 \leftarrow [R5]$):

1. SELA encodes “R5” — R5’s output-enable goes high (R5 selected to drive the bus).
2. R5’s contents are driven onto the common bus.
3. SELD encodes “R3” — R3’s load-enable goes high (R3 selected as destination).
4. On the clock edge, R3 captures whatever the bus holds: $R3 \leftarrow [R5]$.

No direct R5→R3 wire exists; every transfer uses the single common bus plus the two select signals — exactly the mechanism of the lecture’s $R1 \leftarrow [R2]$ example, with R5 in place of R2 and R3 in place of R1.

Q6. For ADD A (accumulator machine):

Phase	Register activity	Memory / ALU	State
Fetch	MAR←[PC]; MDR←instr.; IR←[MDR]; PC←[PC]+1	read instruction word	instruction in IR
Decode	IR opcode $\rightarrow$ control unit	—	operation identified as ADD
Execute	operand [A] fetched; AC←[AC]+[A]	read operand; ALU adds	result in AC

The result is stored in the **accumulator AC** — the implicit destination of the 1-address format. The fetch row is identical to every other instruction; only the execute row is opcode-specific.